1. Create Student class with data members
RollNo
Name
Course
and implement following
• Default constructor
• Parameterized constructor
• Destructor
• Function to accept data
• Function for display data
• Write main function and create array of students pointer and store data of multiple students in array.
• Write Function to find student by Roll No

```
#include<iostream>
#include<string>
using namespace std;

class student {
private:
   int rollno;
   string name;
   string course;

public:
   student() { // default constructor
       rollno = 0;
       name = "";
       course = "";
   }

   student(int r, string n, string c) { // parameterized constructor
       rollno = r;
       name = n;
       course = c;
   }

   ~student() { // destructor
       cout << "Student object for Roll No " << rollno << " destroyed."
<< endl;

   }

   void accept() {
```

```
        cout << "Enter roll number: ";
        cin >> rollno;
        cout << "Enter name: ";
        cin >> name;
        cout << "Enter course: ";
        cin >> course;
    }

    void display() {
        cout << "Rollno: " << rollno << endl;
        cout << "Name: " << name << endl;
        cout << "Course: " << course << endl;
    }

    int getRollNo() {
        return rollno;
    }
};

// Function to search student by roll number
void findstudent(student *arr[], int n, int r) {
    for (int i = 0; i < n; i++) {
        if (arr[i]->getRollNo() == r) {
            cout << "\nStudent found: " << endl;
            arr[i]->display();
            return;
        }
    }
    cout << "\nStudent not found" << endl;
}

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;

    student* arr[n]; // Array of student pointers

    for (int i = 0; i < n; i++) {
        arr[i] = new student();
        arr[i]->accept();
    }
```

```
    cout << "\nStudent details:\n";
    for (int i = 0; i < n; i++) {
        arr[i]->display();
        cout << endl;
    }

    int roll;
    cout << "Enter roll number to search: ";
    cin >> roll;
    findstudent(arr, n, roll);

    // Free memory
    for (int i = 0; i < n; i++) {
        delete arr[i];
    }

    return 0;
}
```

```
compilation terminated.
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$ g++ Time.cpp
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$ ./a.out
Enter first time:
Enter hours: 1
Enter minutes: 20
Enter seconds: 50
Enter second time:
Enter hours: 2
Enter minutes: 45
Enter seconds: 12
Time object destroyed.

Sum of times:
Time: 4h 6m 2s
Time object destroyed.
Time object destroyed.
Time object destroyed.
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$
```

3. Create Date class with data members
dd
mm

Assignment No : 5
Submission Date: 6/3/2026

yyyy

and implement following

- Default constructor
- Parameterized constructor
- Destructor
- Function to find out difference between two dates
- Function to compare dates
- Separate header and implementation files
- Overlaod ==, !=, +, - operator

```
#ifndef DATE_H
#define DATE_H

class Date {
private:
   int dd, mm, yyyy;

public:
   // Constructors & Destructor
   Date();                          // Default: 1/1/2000
   Date(int d, int m, int y);       // Parameterized
   ~Date();

   // Logic Functions
   void display();
   bool isLeapYear(int y);
   int daysInMonth(int m, int y);
   int compare(Date d2);            // Helper for operators

   // Overloaded Operators
   bool operator == (Date d2);
   bool operator != (Date d2);
   Date operator + (int daysToAdd);
   int operator - (Date d2);        // Difference in days
};

#endif
```

```
#include "Date.h"
#include <iostream>

using namespace std;

// 1. Constructors
Date::Date() : dd(1), mm(1), yyyy(2000) {}
```

```
Date::Date(int d, int m, int y) : dd(d), mm(m), yyyy(y) {}

Date::~Date() {}

// 2. Calendar Logic
bool Date::isLeapYear(int y) {
   return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
}

int Date::daysInMonth(int m, int y) {
   int days[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
   if (m == 2 && isLeapYear(y)) return 29;
   return days[m];
}

void Date::display() {
   cout << dd << "/" << mm << "/" << yyyy << endl;
}

// 3. Comparison Logic
int Date::compare(Date d2) {
   if (yyyy != d2.yyyy) return (yyyy > d2.yyyy) ? 1 : -1;
   if (mm != d2.mm) return (mm > d2.mm) ? 1 : -1;
   if (dd != d2.dd) return (dd > d2.dd) ? 1 : -1;
   return 0;
}

// 4. Operator Overloading
bool Date::operator == (Date d2) {
   return (dd == d2.dd && mm == d2.mm && yyyy == d2.yyyy);
}

bool Date::operator != (Date d2) {
   return !(*this == d2);
}

Date Date::operator + (int daysToAdd) {
   Date temp = *this;
   for (int i = 0; i < daysToAdd; i++) {
       temp.dd++;
       if (temp.dd > daysInMonth(temp.mm, temp.yyyy)) {
           temp.dd = 1;
```

```
            temp.mm++;
        }
        if (temp.mm > 12) {
            temp.mm = 1;
            temp.yyyy++;
        }
    }
    return temp;
}

int Date::operator - (Date d2) {
    Date dStart = d2;
    Date dEnd = *this;


    if (this->compare(d2) < 0) {
        dStart = *this;
        dEnd = d2;
    }

    int count = 0;
    while (dStart != dEnd) {
        dStart = dStart + 1; // Increment by 1 day using our + operator
        count++;
    }
    return count;
}
```

```
#include <iostream>
#include "Date.h"

using namespace std;

int main() {

    Date d1(6, 3, 2026);
    Date examDate(15, 3, 2026);

    cout << "Today is: "; d1.display();
    cout << "Exam is on: "; examDate.display();

    // Testing Difference (-)
```

```
    int daysLeft = examDate - d1;
    cout << "Days until exam: " << daysLeft << endl;

    // Testing Equality (==)
    if (d1 == examDate) {
        cout << "The exam is today!" << endl;
    } else {
        cout << "The exam is not today." << endl;
    }

    // Testing Addition (+)
    Date nextWeek = d1 + 7;
    cout << "Date after one week (d1 + 7): ";
    nextWeek.display();

    return 0;
}
```

```
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$ g++ Main.cpp Date.cpp -o TestDate
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$ ./TestDate
Today is: 6/3/2026
Exam is on: 15/3/2026
Days until exam: 9
The exam is not today.
Date after one week (d1 + 7): 13/3/2026
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$
```

4.Create Your own String class (Use class name as MyString) with data members

char * p;
int size;

Allocate memory dynamically for pointer and get size from user.
Implement followings
• Default constructor
• Parameterized constructor
• Copy Constructor
• Destructor for clean-up (use delete compulsory)
• Display function.
• Separate header and implementation files

```
#ifndef MYSTRING_H
#define MySTRING_H

class Mystring
{
```

```
   private:
   char * p;
   int size;

   public:
   Mystring();
   Mystring(int s);
   Mystring(const char * val);
   Mystring(const Mystring& other);
   ~Mystring();

   void display() const;
   void input();


};

#endif
```

```
#include "Mystring.h"
#include <iostream>
#include <cstring> // For strlen and strcpy

// Default constructor
Mystring::Mystring() {
   size = 0;
   p = new char[1]; // Allocate minimum memory for the null terminator
   p[0] = '\0';
}

// Parameterized constructor (using a specific size provided by the
user)
Mystring::Mystring(int s) {
   size = s;
   p = new char[size + 1]; // +1 to accommodate the null terminator
'\0'
   p[0] = '\0';            // Initialize as empty string
}

// Parameterized constructor
Mystring::Mystring(const char* str) {
   size = strlen(str);
```

```
    p = new char[size + 1];
    strcpy(p, str);
}

// Copy Constructor (Deep Copy)
Mystring::Mystring(const Mystring& obj) {
    size = obj.size;
    p = new char[size + 1];
    strcpy(p, obj.p); // Copy actual data, not just the pointer address
}

// Destructor for clean-up
Mystring::~Mystring() {
    delete[] p; // Compulsory use of delete[] to avoid memory leaks
}

// Display function
void Mystring::display() const {
    std::cout << p << std::endl;
}

// Function to take string input from the user into the allocated
memory
void Mystring::input() {
    // std::cin >> p works for a single word.
    std::cin >> p;
}
```

```
#include <iostream>
#include "Mystring.h"

int main() {
    std::cout << "--- 1. Default Constructor ---" << std::endl;
    Mystring str1;
    std::cout << "str1: ";
    str1.display();

    std::cout << "\n--- 2. Parameterized Constructor (Direct Text) ---"
<< std::endl;
    Mystring str2("Hello World");
    std::cout << "str2: ";
    str2.display();
```

```
    std::cout << "\n--- 3. Allocate Memory Dynamically & Get Size From
User ---" << std::endl;
    int userSize;
    std::cout << "Enter the maximum size for your new string: ";
    std::cin >> userSize;

    // Call Parameterized Constructor with user size
    Mystring str3(userSize);

    std::cout << "Enter a word (max " << userSize << " characters): ";
    str3.input();

    std::cout << "str3: ";
    str3.display();

    std::cout << "\n--- 4. Copy Constructor ---" << std::endl;

    Mystring str4 = str3;
    std::cout << "str4 (Copied from str3): ";
    str4.display();

    std::cout << "\nEnd of program. Destructors will now clean up
memory!" << std::endl;
    // When main ends, the destructor (~MyString) is automatically
called for str1, str2, str3, and str4.

    return 0;
}
```

```
Date after one week (d1 + 7): 13/3/2026
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$ g++ MainString.cpp Mystring.cpp -o TestString
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$ ./TestString
--- 1. Default Constructor ---
str1:

--- 2. Parameterized Constructor (Direct Text) ---
str2: Hello World

--- 3. Allocate Memory Dynamically & Get Size From User ---
Enter the maximum size for your new string: 5
Enter a word (max 5 characters): ajay
str3: ajay

--- 4. Copy Constructor ---
str4 (Copied from str3): ajay

End of program. Destructors will now clean up memory!
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$
```

5. `#ifndef PLAYER_H`

```
#define PLAYER_H

#include <string>

using namespace std;

class Player {
private:
   string name;
   int age;
   string country;

public:
   // Default constructor
   Player();

   // Parameterized constructor
   Player(string n, int a, string c);

   // Accept function for data input
   void Accept();

   // Display function for data display
   void Display() const;

   // Getters needed for the sorting function
   string getName() const;
   int getAge() const;
};

// Global function to sort the array of Player objects
void SortPlayers(Player arr[], int size);

#endif // PLAYER_H
```

```
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$ g++ MainPlayer.cpp Player.cpp -o TestPlayer
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$ ./TestPlayer
Enter Details for 1 Players

Player 1:
Enter Player Name: ajay
Enter Player Age: 28
Enter Player Country: India

 Unsorted Player List
Name: ajay | Age: 28 | Country: India

 Sorted Player List (By Name, then Age)
Name: ajay | Age: 28 | Country: India
pgcp-ac@cdac-HP-ProDesk-600-G5-PCI-MT:~/cppdemos/Day_5$
```